

第三篇·降阶与代理推理

过桥 C·第 07 讲 POD：把 8192 维流场压成两笔
第 08 讲 代理模型与端侧推理：在浏览器里解流场

《风电场偏航优化与智能可视化自学白皮书》·全 8 册·第 4 册
数据口径：FLORIS 4.6.6 模拟·NREL 5 MW·8 m/s 除非注明（模拟结果，非实测）

本篇内容

- 过桥 C · 从优化到降阶：数据变重了，网页不能变重
- 【第07讲】POD：把 8192 维流场压成两笔（反派：全存）
- 【第08讲】代理模型与端侧推理：在浏览器里解流场（反派：贵如登月）

第三篇 【降阶与代理推理】

本篇覆盖第 07、08 讲。主线问题：**流场太大（8192 个数）、网页太小（26 KB），中间怎么跨过去？**
两个答案：POD 负责「压缩」（07），插值代理负责「查表」（08）。

过桥 C · 从优化到降阶：数据变重了，网页不能变重

第二篇结束时，你手里有一堆「账本」：cases.csv、fields/*.npz
、48 行风玫瑰——但要把流场「画」给评委看，数据必须进浏览器。矛盾出现：**FLORIS 的流场切片是 64×128 网格（8192 个数/片）×13 偏航角 × 多个风速，而静态站零后端、断网可用。**

你将依次遇见	它专治的病
第 07 讲 POD	「流场全存，网页变砖」
第 08 讲 代理推理	「每次现场算，断网就死」

阅读纪律（本篇通用）：

- 1. 先分清「证明」与「上线」：POD 在本项目首先是流场低复杂度的证明，代理插值才是上线的推理通道；
- 2. 每个数字带可信域（6~12 m/s、±30°）；
- 3. 深潜框可跳；章末自测不过关就回心智图。

下一页反派登场，它的名字叫全存。

【第07讲】POD：把 8192 维流场压成两笔（反派：全存）

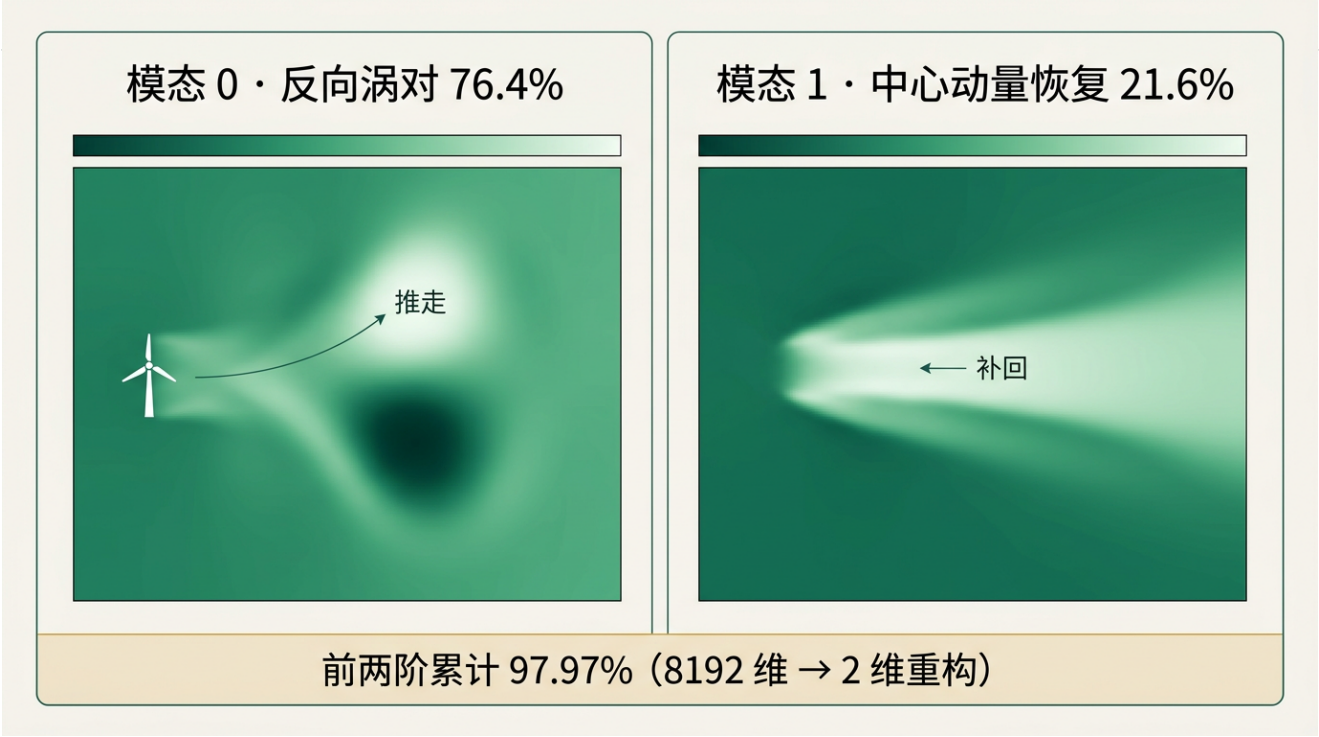
本讲开场 90 秒

上讲闪回（第 06 讲）：风玫瑰算完，下一步是把流场「搬上」网页。

本讲反派：「全存」——直觉认为「流场要展示，就得把 8192 个数全存下来」。

只要带走 3 句口诀：

- 1. SVD = 找两笔最重要的「笔画」：按能量排序，前两笔就够。
- 2. 模态 0 推走（76.4%）、模态 1 补回（21.6%）：尾流的两个动词。
- 3. 2 模态 = 97.97%：剩下 8190 维只占 2% [README / AUDIT A]。



看图三秒：左联深绿浅绿「被推歪」= 模态 0；右联中心变浅 = 模态 1「补回」。尾流的物理，全在这两个动词里。

本讲赌注：97.97% 不只是数字，它是「尾流是低复杂度系统」的物理证明——代理模型可行的前提。

1. 剧场：向陌生人描述你的好朋友，你不会列 8192 个面部坐标——你会说「圆脸 + 单眼皮」，对方脑海里已经有了 90% 的形象。
- POD 就是这件事的数学化：把任意流场分解成一组「正交基本图案」（模态），按能量排序，取前几个，流场就能重构。

描述元素	POD 元素
------	--------

描述元素	POD 元素
8192 个面部坐标	64×128 流场网格（8192 维）
两个词「圆脸+单眼皮」	模态 0 + 模态 1
像了 90%	能量占比 97.97%

先猜后揭：模态 0 是「推走」还是「补回」？揭晓：推走——反对称偶极子（CVP）能量最大（76.38%），它正是第 02 讲「把尾流推歪」的那个物理动作的「模态画像」[AUDIT A：模态 0 订正为反对称偶极子]。

2. 工程死穴：网页的「数据重量」问题

- 13 偏航角 × 8192 数 × 浮点 ≈ 每偏航角数百 KB 量级（实际以 npz 切片存留档）；
- 网页零后端，数据必须「注入」JS 文件，离线包要小；
- 若「全存」：首屏重、断网包大、维护面广；
- POD 证明流场的「模式」可以 2 个系数重构——网页只需要「查哪两个笔画、各占多少」。

定位声明（诚实边界）：现役 [pod.html](#) 的作用是展示 POD 分解结果（3 张 Nature 图同排）+ 证明流场可压缩性；现役网页的流场渲染走的是「关键切片注入 + 插值」通道（[data_3d.js](#)，5 偏航 × 9 高度层），POD 系数尚未接进在线重构控制回路 [README 页面表 / HANDOFF]。

3. 数学公式从 0 到 1：矩阵的「两笔」

先用脸的话说一遍：把 8192 维流场摊成 64 行 × 128 列的矩阵，SVD 把它「摊平」成一组正交笔画。

3B. 本讲符号卡

符号	人话	备注
U (64×128)	流场矩阵	一个偏航角的速度场切片
σ_i	第 i 笔的「粗细」	$\sigma_1 \geq \sigma_2 \geq \dots$
φ_i	第 i 笔的「形状」	空间模态
$E_i = \sigma_i^2 / \sum \sigma^2$	第 i 笔的能量占比	模态 0 76.38%

3C. 主路径只要两条式

核心式 ① — SVD 分解 [式] $U = \sum_i \sigma_i \cdot \varphi_i \cdot \psi_i^T$ ($\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n$) 回译：每个 σ_i 对应一个「图案」（空间形状 φ_i × 强度 ψ_i ）； σ 越大，这个图案越「重要」。

核心式 ② — 能量占比 [式] $E_i = \sigma_i^2 / \sum_i \sigma_i^2$ 回译：「那一笔占多少能量」。模态 0 76.38%，模态 1

- 为什么能量是 σ^2 不是 σ ？流场总能量 = 所有网格点速度（平方）之和；SVD 的正交性保证每个模态的能量贡献恰好 = σ_i^2 。- 模态 0 的物理身份：反对称——上半区均值 +0.0058、下半区均值 -0.0056（上下反对称），即反向旋转涡对（CVP）把尾流推偏（与第 02 讲「推走」同一物理源头）[AUDIT A：独立 SVD 重算订正]。- 一段事故史：网页曾写「82.4/15.7/累

计 98.1%」——与自家配图标题（76.4/21.6）互相矛盾；审计独立重算 SVD 后订正，模态 0 身份一并订正为反对称偶极子 [AUDIT A]。本讲的数字值得被强调：它们是「打」出来的。

4. 值班手册

问「POD 是什么」→ 流场矩阵的奇异值分解：模态按能量排序，前几个重构流场
 问「凭什么 2 个模态够」→ 97.97%，剩余 2% 是流场的「细噪」
 问「两个模态是什么」→ 模态 0 反向涡对「推走」（76.4%）；模态 1 中心动量「补回」（21.6%）
 问「用在哪」→ pod.html 展示 + 「流场低复杂度」证明（代理模型可行的前提）

一句话记住：8192 个数 → 2 个数，尾流不是「乱」的，是「有规律的舞」。

5. 比分日

先报胜负：前两阶累计 97.97%，独立 SVD 重算与图一致 [AUDIT A]。

模态	能量占比	物理身份
模态 0	76.38%	反对称反向涡对（CVP）「推走」
模态 1	21.58%	中心动量「补回」（卷吸恢复）
前两阶累计	97.97%	流场可重构
其余	2.03%	高阶细节

怎么读这张表：「身份」列比「数字」列更重要——前两模态有清晰物理身份（推/补），这正是 POD「可解释」而非「黑箱压缩」的原因。

6. 可信度

维度	打分	人话
方法	5	SVD 成熟数学工具
数据	5	独立重算
范围	3	单截面（64×128）、单工况稳态；时程/三维未覆盖 [软肋④]
应用	3	本项目是「展示 + 证明」，非在线降阶控制器

可以攻击的点：① 单快照而非时变（真时变 POD 需要快照序列）；② 单工况（8 m/s 270°），其他工况模态能量占比需重算；③ POD 系数未接入控制回路（申请书的 PINN 路线是待办，C 级）。承前：第 02 讲 CVP「推走」获得了模态画像 → 启后：第 08 讲——除了「压缩」，还有一条更便宜的「查表」路。

附

POD 两笔模态 0 推走 76.38%（CVP 偶极子） | 模态 1 补回 21.58%（中心恢复）前两阶累计 97.97% | 8192 维 → 2 维 旧错值 82.4/15.7/98.1 已订正 [AUDIT A]——数字是打出来的。

章末：30 秒复述·掌握门自测

30 秒复述稿（可朗读）：POD = 流场矩阵 SVD，模态按能量排序：模态 0 反向涡对「推走」76.38%，模态 1 中心动量「补回」21.58%，前两阶 97.97%；它证明尾流低复杂度、代理可行；旧错值 82.4 被审计订正。反派「全存」，破绽「两笔就够」。

2. 模式 0 = CVD 偶极了「排布」（后对称）、模式 1 = 中心轴量「相同」（半对称）、上篇 02 讲 CVD 信标

3. 证明了「该工况流场是低复杂度的、可 2 模态重构」；不能证明「所有工况都 97.97%」，也不能证明「时变尾流也低复杂度」。

元数据：证据等级 A（独立 SVD 重算） | 复算路径 [pod_analysis.py/ pod_results/](#) | 关联页 [pod.html](#)。

【第08讲】代理模型与端侧推理：在浏览器里解流场（反派：贵如登月）

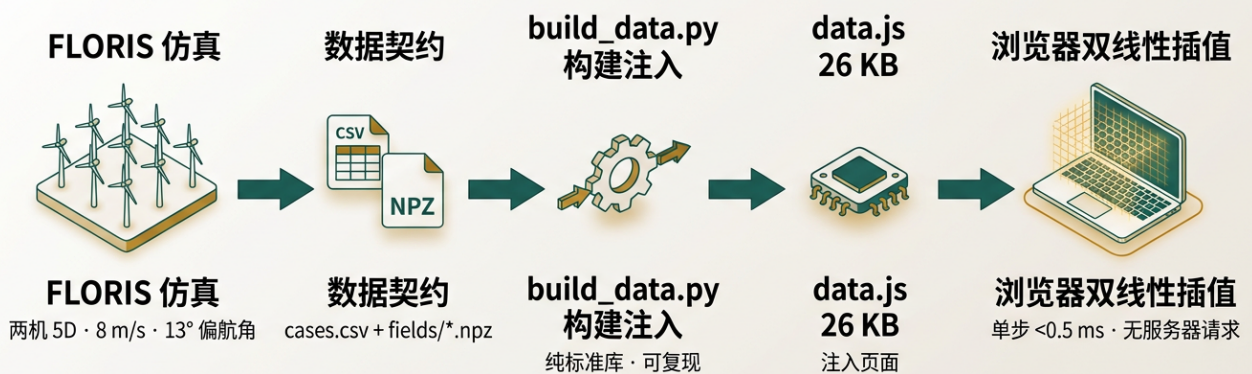
本讲开场 90 秒

上讲闪回（第 07 讲）：POD 证明流场「可压缩」；但网页真正的诉求是「滑块一动，立刻有功率」。

本讲反派：「贵如登月」——直觉认为「跑流场要服务器、要 GPU、要 API」。

只要带走 3 句口诀：

1. 数据在 build 时注入，运行时只查表 + 插值：服务器是「夜班工人」。
2. 26 KB + <0.5 ms：整个流场数据库比一张网页还小 [AUDIT 六 / 口袋卡]。
3. 可信域是生命线：6~12 m/s、 $\pm 30^\circ$ ，越界自动提示，绝不硬答 [README]。



零后端 · 离线可用 · 无第三方接口请求

本讲赌注：「零后端」不是炫技，是价值选择——答辩断网备胎、审计可复现，都站在它头上。

1.

剧场：高档餐厅不会每单现熬汤——厨师提前熬好 20 种基础酱汁（FLORIS 预计算 13 偏航 × 4 风速），服务员（浏览器）只做「取 1/3 酱汁 A + 1/3 酱汁 B」（双线性插值），0.5 秒上桌。

餐厅元素	工程元素
基础酱汁	预计算切片（13×4 网格）
1/3 A + 1/3 B	双线性插值
服务员	浏览器 interp.js
后厨打烊	零后端

先猜后揭：插值的误差是「模型误差」吗？揭晓：不是——插值没有「模型误差」，只有「网格粗细」（第 04 讲的前提）。[model.html](#)页面上的 XGBoost 才是「模型」，而它的证据只有图源级（下文诚实声明）。

2. 工程死穴：答辩的「断网」

答辩现场、组会现场、展厅：网络可能断、服务器可能不在、GPU 可能是奢侈品。

团队的选择：把数据「印」进网页（data.js），浏览器就是计算机：

- [site/build_data.py](#)（纯标准库）把 CSV/JSON 注入 JS [README] ；
- 浏览器 [interp.js](#) 双线性插值 <0.5 ms [口袋卡] ；
- Plotly.js / three.js 全部本地离线依赖，运行时不请求任何第三方接口 [HANDOFF 五] 。

这就是「离线可用」的由来——也是审计能做「位级复现」的原因

：网页里的数字与源数据是同一份文件。

3. 数学公式从 0 到 1：契约即接口法

3B. 本讲符号卡

符号	人话	备注
predict_power	AI 组契约函数	输入 2 数，输出 4 字段
outOfRange / reasons	两个「诚信字段」	越界 = 不回答
s, t	插值权重	小矩形定位

3C. 主路径只要两条式

核心式 ① — 功率契约〔式〕 $\text{predict_power}(\text{yaw}, U) \rightarrow \{p_1, p_2, \text{ptot}, \text{outOfRange}, \text{reasons}\}$ 回译：

输入两个数（偏航、风速），输出两机功率 + 两个诚信字段：outOfRange（是否越界）

与 reasons（为什么）——API 本身把「诚实」写进了接口 [README 数据契约] 。

核心式 ② — 插值（呼应第 04 讲）〔式〕 $f(\text{yaw}, U) = (\text{yaw}, U)$ 所在小矩形四角按 s,t 加权 回译：

- 数据为什么只有 26 KB ? 注入的是「功率表 + 关键切片 + 风玫瑰 48 行」, 不是全部 8192 维切片; 3D 页单独走 data_3d.js (5 偏航 × 9 高度层关键切片) [README / AUDIT 六] 。流场 npz (64×128×13) 是「留档资产」, 网页取「关键视角」。 - XGBoost 的证据边界: model.html 展示 XGBoost 交付图源, 但原始训练压缩包未纳入仓库, 不承诺仓内完整重训 [README 数据口径 / AUDIT F] 。这就是「边界诚实」——不能复现的模型, 只配「图源级佐证」。 - 留档工具偏差: Streamlit 留档 predict_power_2d 曾用 cos^1.5 启发式 (口径应为 FLORIS 原生 1.88), 审计仅加注释不改行为 (留档工具不影响站点 JS 口径) [AUDIT 六] 。

4. 值班手册 (三组数据契约)

```
仿真组: cases.csv / fields/*.npz → u(64,128) m/s ( 绝对风速, 不归一化 )
控制组: find_yaw_for_target(target, U) → (best_yaw, P, err%)
AI 组: predict_power(yaw, U) → {p1, p2, ptot, outOfRange, reasons}
字段形状不可变更 [ README 数据契约 ]
```

一句话记住: 契约是五人协作的「接口法」——你只需承诺形状, 内部随便演化。

5. 比分日

先报胜负: 数据管道全链路纯标准库可复现, build_data.py 重复构建哈希不变 [HANDOFF 审计结果] 。

项	数值	来源
data.js 体积	26,502 B (≈ 26.4 KB)	AUDIT 六
单步插值	<0.5 ms	口袋卡
可信域	6~12 m/s、±30°	README
离线依赖	Plotly / Three.js / Geist 本地	HANDOFF

体感换算: 「服务器」成本 = 0 ; 「夜班」成本 = 跑一次 build_data.py 。网页每个数字想质疑, 三步回到 CSV (页面 → data.js → cases.csv) 。

6. 可信度

维度	打分	人话
架构	5	零后端、位级自治
契约	4	字段显式、越界提示内建
XGBoost	2	图源级佐证, 仓内不可重训 [软肋⑤]
泛化	3	可信域边界清晰; 域外「不答」不「猜」

可以攻击的点: ① XGBoost 不可复现 (原始压缩包不在仓); ② 网格 13×4、线性插值 (无曲率修正); ③ 「26 KB」是双机口径, 九机流场切片走 data_3d.js 单独通道。承前: 第 04 讲插值曲面 + 第 07 讲可压缩性 → 启后: 第四篇——除了「每次搜最优角」, 能不能让机器「学会」跟踪?

端侧推理 build_data.py (纯标准库) → data.js 26.4 KB → 浏览器 <0.5 ms 越界 = 不回答 (outOfRange

/ reasons) | XGBoost = 图源级佐证，别夸

章末：30 秒复述 · 掌握门自测

30 秒复述稿（可朗读）：代理与端侧：数据 build 时注入（26.4 KB），浏览器只查表 + 双线性插值（<0.5 ms），可信域 6~12 m/s、 $\pm 30^\circ$ ，越界自动拒答
；XGBoost 只有图源级佐证。反派「贵如登月」，破绽「夜班工人一次，白天零成本」。

自测（先做再对答案）：

1. `predict_power`输出的两个「诚信字段」是什么？它们的作用？

2. 网页凭什么「断网可用」？依赖哪三类本地文件？

3. XGBoost 为什么是「图源级佐证」？它与 FLORIS 数据的证据差别在哪？

»

3. 原始训练压缩包不在仓库、无法仓内重训，只有交付图可看；FLORIS 数据有完整生成脚本可复算（A 级）。

元数据：证据等级 A（管道）+ B/C（XGBoost 图源） | 复算路径 `site/build_data.py` + `site/check_contract.py`
| 关联页 model.htmlinterface.html。